Mikhail Sinkin

Novi Sad, Serbia (UTC+2)
sinkin.m@gmail.com |+381629379731 | https://www.linkedin.com/in/mikhail-sinkin

---

Testing Approach

Requirements Analysis

The first step was identifying the available functional requirements that could serve as a baseline for testing activities. Since no dedicated specification was provided, the system behavior is derived from available users documentation, located at /about and /faq pages, which described the system's functionality and implied user workflows.

Each statement or described capability in this documentation was treated as an individual atomic requirement. Based on this, corresponding verification points were prepared and organized into a structured checklist.

As for UI/UX and data table validation, the testing was based on standard UI/UX quality heuristics rather than product-specific business rules. Validation criteria included areas such as: sorting behavior, filtering, pagination, search consistency, column interaction, state persistence, and general usability expectations were derived from commonly accepted industry best practices for modern web applications.

Test Design Strategy

The initial test coverage was implemented in the form of functional checklists grouped by feature areas: authentication, browsing catalog, UI/UX and table view, cart management, payment, owned assets, profile and account management.

The testing process combined structured checklist-based validation and exploratory testing techniques: the set of checklists was evolved iteratively during execution as new behaviors discovered or additional edge cases come to mind during testing.

Scope and Constraints

Given the format and scope of the assignment, checklist-based coverage was considered the most practical and efficient approach for achieving broad functional validation within test task assessment context.

For a subsequent testing iteration, the next logical step would be: extracting a dedicated critical-path regression suite, grouping scenarios by business risk, and introducing priority-based test structuring.

Test Cases

In addition to checklist-based coverage, six detailed test cases were prepared as examples of structured scenario documentation.

Testing Results

During the testing process, 27 defects of varying severity levels were identified across the system:

- Critical: 7
- Major: 15
- Minor: 3
- Trivial: 2

Critical findings indicate structural problems in key business flows. The most severe issues impact billing logic consistency, cart and payment functionality, and financial data correctness between frontend and backend layers. In several cases, the system allows invalid states that result in inconsistent cart and asset ownership states, enables account balance manipulation, and exposes restricted data.

Defects were documented using a GitHub Project-based tracking workflow with severity categorization.

References

Readme file

Github Project Board — discovered issues

Checklists

Test Cases